AP Computer Science A Study Guide

Key Concepts Checklist (Static vs Non-Static #1) + 11 Common Compile Errors

PART 1: KEY CONCEPTS CHECKLIST

[ ] 01. Static vs. Non-Static (Class vs. Instance) — TOP PRIORITY

static belongs to the Class as a whole with ONE shared copy in memory. Non-static belongs to an individual object created with new.

- static methods/fields are called via ClassName.method() or ClassName.field
- Non-static methods/fields are called via objectName.method()
- static methods CANNOT use "this" or access instance fields directly
- Calling an instance method from static main without creating an object causes "non-static method cannot be referenced from a static context"

```
public class Student {
  private String name;           // Instance var (unique per student)
  private static int count = 0;  // Static var (shared across all students)
  public Student(String n) { name = n; count++; }
  public static int getCount() { return count; }
```

[ ] 02. Inheritance, super(), & Polymorphism (Dynamic Binding)

Subclasses inherit superclass methods via extends. super() calls the superclass constructor and MUST be the first line of the subclass constructor.

- Compile-time check: Compiler verifies method exists on declared reference type
- Runtime execution: Java runs the overridden method on the actual instantiated object class (Dynamic Binding)
- Subclasses cannot access private superclass fields directly; must use getters/setters or super()

```
Animal a = new Dog(); // Polymorphic declaration
a.speak();            // Runs Dog.speak() at runtime
// a.fetch();         // CE: Animal reference does not declare fetch()
```

[ ] 03. References, Aliasing, & == vs. .equals()

Primitives store raw values. Objects store memory addresses. == on objects checks memory location identity, NOT character content equality.

- Always use .equals() to compare String contents
- Aliasing: Dog d2 = d1; does NOT create a second dog; both variables point to the same object in heap memory
- Strings are immutable; methods like .toLowerCase() return a NEW String and never modify the original

```
String s1 = new String("hi");
String s2 = new String("hi");
System.out.println(s1 == s2);     // false (different heap addresses)
System.out.println(s1.equals(s2)); // true  (same characters)
```

[ ] 04. 1D Arrays vs. ArrayList<E> & The Backward Removal Loop

Arrays have fixed capacity (arr.length, field). ArrayList has dynamic size (list.size(), method) and stores wrapper objects.

- Arrays: arr[i] | ArrayList: list.get(i), list.set(i, val)
- Removing elements in a forward loop shifts indices, skipping consecutive matches
- Always traverse backwards (for int i = list.size() - 1; i >= 0; i--) when removing elements

```
// Safe removal loop:
for (int i = list.size() - 1; i >= 0; i--) {
  if (list.get(i).equals("remove")) list.remove(i);
}
```

[ ] 05. 2D Arrays & Row-Major Traversal

2D arrays are arrays of arrays stored in row-major order: grid[row][col]. Number of rows = grid.length, number of cols = grid[0].length.

- Outer loop indexes rows (r = 0; r < grid.length; r++)
- Inner loop indexes columns (c = 0; c < grid[r].length; c++)

```
int[][] matrix = new int[2][3]; // 2 rows, 3 columns
for (int r = 0; r < matrix.length; r++)
  for (int c = 0; c < matrix[r].length; c++)
    matrix[r][c] = r * 10 + c;
```

[ ] 06. Recursion & Base Case Tracing

A recursive method calls itself with simpler input until reaching a base case. Missing or unreachable base cases trigger StackOverflowError.

- Code before recursive call executes on the way DOWN the stack
- Code after recursive call executes on the way BACK UP the stack

```
public static int mystery(int n) {
  if (n <= 1) return 1; // Base case
  return n + mystery(n - 1); // mystery(4) = 4+3+2+1 = 10
}
```

[ ] 07. Search & Sort Algorithms (Big-O & Mechanics)

AP CSA requires understanding Sequential Search, Binary Search, Selection Sort, Insertion Sort, and Merge Sort.

- Binary Search (O(log n)) REQUIRES SORTED DATA! Halves search space each step
- Selection Sort (O(n^2)): Repeatedly finds minimum and swaps to front (minimum swaps)
- Insertion Sort: O(n) best / O(n^2) worst (fast if mostly sorted)
- Merge Sort (O(n log n)): Recursive divide-and-conquer; merges sorted halves

```
// Binary search middle formula: mid = (low + high) / 2;
// If target < arr[mid], search left: high = mid - 1;
```

[ ] 08. Boolean Logic, Short-Circuiting, & De Morgan's Laws

!(A && B) is equivalent to (!A || !B). !(A || B) is equivalent to (!A && !B). Short-circuit evaluation stops compound checks early.

- Negation of < is >= (must include equality!)
- Short-circuit: in (s != null && s.length() > 0), if s is null, s.length() is never called

```
// Safe null check with short-circuit &&:
if (str != null && str.length() > 0) { ... }
```

PART 2: 11 COMMON COMPILE ERRORS (CE) GUIDE

01. cannot find symbol

Definition: The compiler encounters an identifier (variable, method, or class) that has not been declared or is out of scope.

'L BROKEN (Causes CE)

```
for (int i=0; i<5; i++) { int secret = 42; }
System.out.println(secret); // CE: cannot find symbol
```

' FIXED SOLUTION

```
int secret = 0; // Declare before loop
for (int i=0; i<5; i++) { secret = 42; }
System.out.println(secret); // OK: prints 42
```

02. incompatible types: possible lossy conversion from double to int

Definition: Attempting to store a floating-point number (double) into an int variable without an explicit cast.

'L BROKEN (Causes CE)

```
int count = 5.8;     // CE: lossy conversion
int half = 10 / 4.0; // CE: 4.0 makes result double
```

' FIXED SOLUTION

```
int count = (int) 5.8;       // Truncates to 5
int half = (int) (10 / 4.0); // Truncates to 2
```

03. variable <var> might not have been initialized

Definition: Reading or printing a local method variable before Java can guarantee an initial value on every branch.

'L BROKEN (Causes CE)

```
int grade; int score = 75;
if (score >= 90) grade = 1;
System.out.println(grade); // CE: uninitialized
```

' FIXED SOLUTION

```
int grade = 0; // Safely initialized
int score = 75;
if (score >= 90) grade = 1;
System.out.println(grade); // OK
```

04. missing return statement

Definition: A non-void method promised to return a value, but there is a path where execution finishes without returning.

'L BROKEN (Causes CE)

```
public boolean isPositive(int n) {
  if (n > 0) return true;
  // CE: missing return statement if n <= 0
}
```

' FIXED SOLUTION

```
public boolean isPositive(int n) {
  return n > 0; // Concise and guaranteed
}
```

05. unreachable code / unreachable statement

Definition: Statements placed directly after a line that unconditionally exits the method or loop (like return or break).

'L BROKEN (Causes CE)

```
public int getVal() {
  return 10;
  System.out.println("Done"); // CE: unreachable
}
```

' FIXED SOLUTION

```
public int getVal() {
  System.out.println("Done");
  return 10; // Placed at exit
}
```

06. non-static method cannot be referenced from a static context

Definition: Calling an instance method directly from static main without creating an object instance first.

'L BROKEN (Causes CE)

```
public class Game {
  public void play() { ... }
  public static void main(String[] args) {
    play(); // CE: non-static method in static context
  }
```

' FIXED SOLUTION

```
public static void main(String[] args) {
  Game g = new Game();
  g.play(); // OK: called on instance g
}
```

07. array required, but ArrayList found

Definition: Using array brackets [ ] on an ArrayList, or calling .get() / .size() on a basic 1D/2D array.

'L BROKEN (Causes CE)

```
ArrayList<String> list = new ArrayList<>();
list.add("Java");
System.out.println(list[0]); // CE: array required
```

' FIXED SOLUTION

```
System.out.println(list.get(0)); // OK: prints Java
System.out.println(list.size());  // OK: prints 1
```

08. constructor in class cannot be applied to given types

Definition: Calling new ClassName(...) or super(...) with parameters that do not match any constructor.

'L BROKEN (Causes CE)

```
class Student { public Student(String name) {} }
Student s = new Student(); // CE: constructor mismatch
```

' FIXED SOLUTION

```
Student s = new Student("Franky"); // OK: matches
parameter
```

09. bad operand types for binary operator (<, >, ==)

Definition: Applying arithmetic or relational operators to objects (like Strings) that do not support them.

'L BROKEN (Causes CE)

```
String s1 = "apple"; String s2 = "banana";
if (s1 < s2) { ... } // CE: bad operand types
```

' FIXED SOLUTION

```
if (s1.compareTo(s2) < 0) {
  System.out.println(s1 + " comes before " + s2);
}
```

10. 'void' type not allowed here

Definition: Trying to print, assign, or pass the return value of a method that returns void (no value).

'L BROKEN (Causes CE)

```
ArrayList<String> list = new ArrayList<>();
System.out.println(list.clear()); // CE: void not allowed
```

' FIXED SOLUTION

```
list.clear();                    // Separate statement
System.out.println("Cleared!"); // OK
```

11. class, interface, or enum expected

Definition: An unexpected closing curly brace } closed the class prematurely, leaving code floating outside.

'L BROKEN (Causes CE)

```
public class Demo {
  public static void main(String[] args) {}
} // Extra brace ends class!
  public void helper() {} // CE: class expected
```

' FIXED SOLUTION

```
public class Demo {
  public static void main(String[] args) {}
  public void helper() {} // Properly enclosed
}
```